

Get Out

Get Out is an addictive casual endless game designed for mobile and desktop platforms. The objective is simple yet challenging—shoot the ball out of the central zone without hitting the moving obstacles. Each successful escape earns you points, and as the game progresses, new obstacles appear, making it increasingly difficult.

Features:

- ✓ **Endless Gameplay** – The game dynamically generates new obstacles, ensuring unlimited replayability.
- ✓ **Simple One-Tap Controls** – Easy-to-learn mechanics with a single-tap shooting system.
- ✓ **Challenging Obstacles** – Various moving barriers keep players engaged and test their timing skills.
- ✓ **Scoring & Progression** – Earn points for each successful shot, with increasing difficulty over time.
- ✓ **Minimalist & Smooth UI** – Clean and modern design with intuitive controls.
- ✓ **Optimized Performance** – Lightweight and well-optimized for smooth performance on all devices.
- ✓ **Full Source Code** – Well-structured and commented code, making customization easy.

Technical Specifications

- **Unity Version:** Compatible with Unity **2022.3.56f1**
- **Render Pipeline Support:** Built-in
- **Physics Engine:** Uses Unity's default **2D Physics Engine** (supports custom colliders).
- **Dependencies:** **TextMeshPro** is required (for UI).

Project Contents

1. Audio

- Contains all audio files used in the game.
- Examples: Background music, sound effects, voice-overs.

2. Document

- Stores documentation related to the project.
- Examples: Design documents, technical specifications, guides.

3. Fonts

- Contains font assets used in the game UI.

4. Graphics

- Contains graphical assets such as textures, sprites, and materials.
- **2D**: Stores 2D sprites, icons, and textures.
- **Material**: Contains materials and shaders for rendering.
- **2D**: Stores 3D graphics fbx files

6. Prefabs

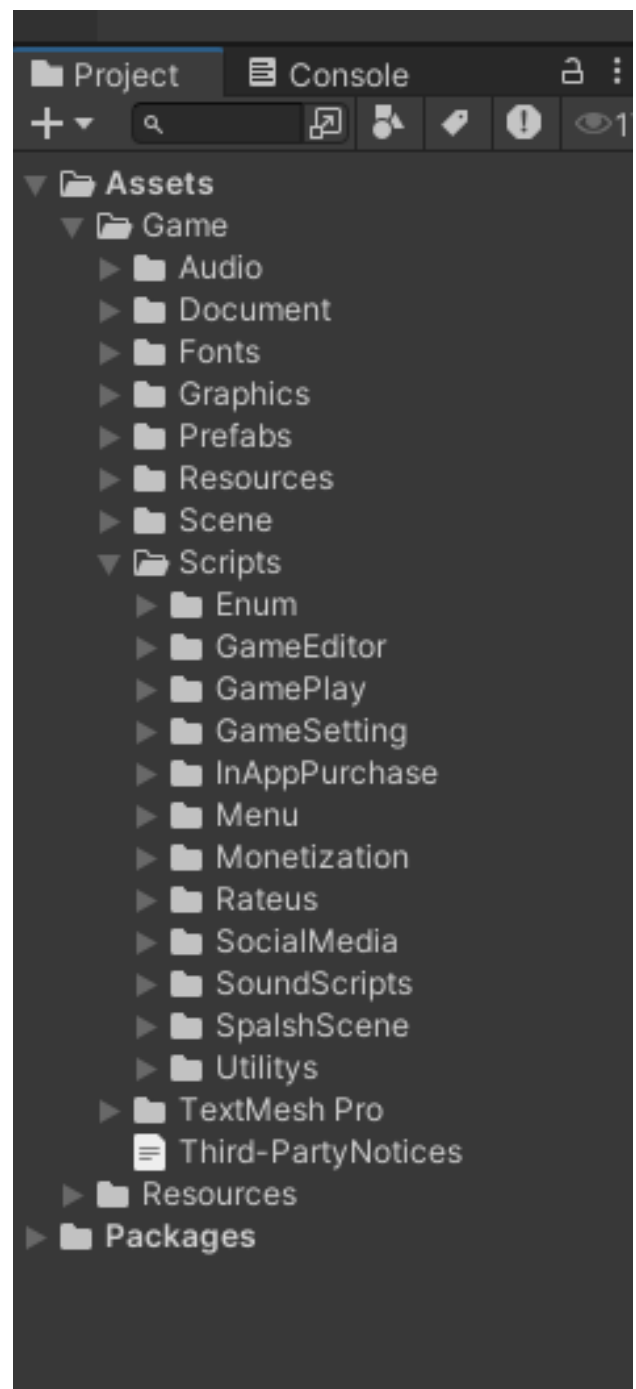
- Stores reusable game objects and UI elements.
- Examples:
 - Game item prefabs
 - Particle effect prefabs

7. Resource

- It contains scriptable objects that store settings for various features.
- Examples:
 - Sound settings
 - Monetization settings

8. Scenes

- Stores Unity scene files.
- Examples:
 - Splash screen scene



- Game logic and mechanics.
- Examples: In-game scenes

9. Scripts

- Contains all the scripts related to th
 - Player movement scripts
 - UI behavior scripts
 - Game mechanics scripts

Reskinning a Merge Item - Guide

1. Locate and Select the Prefab:

- Navigate to **Assets/Game/Prefab/Gameplay.**
- Select the prefab you want to modify.

2. Create a new Level of Obstacle

- Create an Empty game object in the scene and name it properly
- Add a Child game object with a 2D sprite as you like and add a 2D collider based on their shape example Circle collider or, Box Collider and for complex shapes add a polygon collider
- Adjust the scale and position.
- Then put that under level hander and add its reference to levels in level handler script as given in script

